

Tehnička dokumentacija - Aetherfront: Zeppelin Wars

Verzija: 2.1

Datum: 2. srpnja 2026.

Status: release kandidat s prvim završnim balance passom, steampunk HUD-om, SFX-om, glazbom i macOS paketom

Arhitektura

Paket **aetherfront** koristi Python 3.12 i PyGame. Ulazna točka **python -m aetherfront** stvara objekt **Game**, koji upravlja inicijalizacijom PyGamea, prozorom, glavnom petljom, crtanjem i sigurnim gašenjem.

Prikaz se crta na internu površinu veličine 640×360 i skalira na prozor veličine 1280×720. Petlja je ograničena na 60 slika u sekundi. Trenutačna verzija prikazuje vizualni Mode7 prototip s osnovnim borbenim gameplayem, protivnicima, projektilima, pickupima, HUD-om, dijagnostičkim FPS-om, glavnim izbornikom, uputama, pauzom i restartom pokušaja.

Dodani su i suptilni proceduralni efekti za pucanje, štetu, boss pogotke, skupljanje popravka i uništenje protivnika.

Proceduralni parallax sky slojevi crtaju se iznad horizonta sa smanjenim intenzitetom i pomiču se sporije od kamere. Brodovi i popravak sada koriste bogatiji Victorian airship polish: platnene balone, drvene gondole, mjedene nosače, zakovice, kabine i dimnjake. Za završno balansiranje **CombatSession** bilježi vrijeme pokušaja, uništene standardne protivnike, skupljene popravke i ukupno primljenu štetu.

Aktualni HUD koristi gornju horizontalnu minimalističku steampunk traku s mjedenim rubovima, tamnim staklom, segmentiranim health barom i kratkim grupiranim oznakama. Stari gameplay tekst s kontrolama uklonjen je iz scene, dok zasebni ekran uputa ostaje dostupan iz glavnog izbornika.

Audio sloj koristi ElevenLabs MP3 efekte i generirane WAV glazbene loopove učitane preko **pygame.mixer**, uz sigurni fallback bez zvuka ako audio uređaj ili mixer nisu dostupni.

Mode7 projekcija

Mode7Projection predizračunava perspektivu za 224 retka tla ispod horizonta na retku 135. Za redak zaslona **y** udaljenost uzorka računa se formulom:

udaljenost = visina_kamere × žarišna_duljina / (y - horizont)

Žarišna duljina proizlazi iz širine interne slike i horizontalnog vidnog polja od 60°.

Udaljenost se ograničava na 1.400 jedinica. Za svaki stupac zatim se računa bočni otklon, a vektori kamere naprijed i desno pretvaraju udaljenost i otklon u koordinate svijeta. Operator modulo omata obje koordinate unutar svijeta veličine 2.048 jedinica.

Rezultat **ProjectionGrid** sadrži **screen_rows**, **world_x** i **world_y**. Matrice koordinata imaju oblik 224×640 i koristi ih **Mode7Renderer** za NumPy uzorkovanje teksture. U izračunu nema Python petlje po pikselima.

Cloud tekstura terena

load_terrain_texture() zadano učitava **assets/images/terrain/cloud_layer.png**, generirani fotorealistični cloud PNG asset. PyGame ga čita kao RGB površinu, a **pygame.surfarray** ga pretvara u kontinuiranu NumPy matricu tipa **uint8** koju Mode7 renderer može uzorkovati bez promjene projekcijske matematike. Tekstura je kvadratna i koristi svijetle dnevne kumulusne oblake, prirodne plave zračne praznine i mekane sjene kako bi podloga jasnije izgledala kao sloj oblaka ispod broda, a ne kao snijeg, led, voda ili kopno.

generate_terrain_texture(size=512, seed=7) ostaje proceduralni fallback ako PNG asset ne postoji ili se ne može učitati. Fallback zadržava deterministički oblačni uzorak kako bi aplikacija i testovi ostali sigurni i bez asseta. Cloud PNG evidentiran je u oba asset manifesta i uključen u package-data konfiguraciju.

Vizualni renderer

Mode7Renderer pretvara omotane koordinate svijeta u indekse proceduralne teksture. Napredno NumPy indeksiranje uzorkuje sva 143.360 piksela tla odjednom, a **pygame.surfarray.blit_array()** prenosi rezultat na internu površinu. Iznad tla crta se unaprijed izračunata olujno-plava gradacija s parallax slojevima, a mjedena linija odvaja horizont.

Metoda **sample_ground()** odvojena je od crtanja kako bi se rezultat uzorkovanja mogao automatizirati provjeriti. **draw()** prihvaća samo internu površinu 640×360, čime se sprječava tiho rastezanje ili rezanje projekcije.

Prije prijenosa na PyGame površinu **Mode7Renderer** na najudaljenije retke tla primjenjuje svijetli atmosferski haze. Haze se postupno gasi prema donjem dijelu zaslona, pa ublažava perspektivno rastezanje teksture uz horizont bez zatamnjenja fotorealistične cloud podloge, bliskih borbenih objekata ili same Mode7 projekcije.

Na ciljanom M1 MacBook Airu izolirani benchmark višestruko je ostao iznad praga od 55 FPS-a. Završna kratka provjera 2. srpnja 2026. izmjerila je 156,4 FPS-a kroz 12 sekundi, a ranija 60-sekundna provjera 20. lipnja izmjerila je 159,3 FPS-a.

Parallax sky

create_parallax_sky_layers() stvara tri deterministička PyGame RGBA sloja dimenzija 640×136, što odgovara području iznad horizonta. Slojevi su **far_clouds**, **industrial_haze** i **near_streaks**, a svaki ima vlastiti **scroll_factor** i **opacity** za kasniji sporiji pomak u odnosu na kameru. Modul ne koristi vanjske slike; oblici oblaka,

udaljenih balona, lebdećih gradskih silueta, dimnih plumeova i linija nastaju NumPyjem i PyGame površinama.

Mode7Renderer._draw_parallax_sky() prvo crta osnovnu gradaciju, zatim svaki sloj blita uz horizontalno omatanje. Pomak sloja računa se iz položaja kamere i smjera kamere, ali se množi s **scroll_factor**, pa udaljeni oblaci, industrijska izmaglica i bliže linije klize različitim brzinama. Faktori pomaka i alfa vrijednosti namjerno su niski kako bi efekt ostao pozadinski i ne bi odvlačio pozornost od borbe. Parallax ne mijenja tlo, billboard, gameplay, kameru ni HUD.

Billboard projekcija

WorldBillboard povezuje PyGame površinu s položajem i širinom u koordinatama svijeta.

BillboardProjector najprije računa najkraći pomak kroz omotani svijet, a zatim ga rastavlja na dubinu ispred kamere i bočni odmak. Objekti iza kamere, izvan najveće udaljenosti ili potpuno izvan zaslona ne crtaju se.

Vidljivi objekt dobiva **pygame.Rect** čija veličina pada s dubinom. Donji rub pravokutnika prati projekciju ravnine, pa objekt ostaje usidren u svijetu. **project_all()** sortira rezultate od najudaljenijeg prema najbližem kako bi bliži objekti pravilno prekrili udaljene. Sustav je pripremljen za buduće protivnike, projekte i popravke.

Proceduralni Kestrel

create_kestrel_surface() crta transparentnu sliku 96×64 pomoću PyGame elipsi, poligona, linija i krugova. Brod koristi mjedenu, tamnoželjeznu, drvenu, platnenu i cijan paletu te oblikom razlikuje balon, gondolu, kabinu, nosače, zakovice, dimnjak, peraje i pogon. Budući da kamera predstavlja položaj igrača u svijetu, Kestrel ostaje vezan uz zaslon, vodoravno centriran i s donjim rubom na retku 344.

Prikaz je generiran kodom, pa nije dodan vanjski resurs ni zapis u licencne manifeste.

Borbena osnova

data/balance.json trenutačno određuje 100 bodova zdravlja igrača, radijus sudara 18, neranjivost od 1,25 sekundi te zadani radijus 4 i trajanje 3 sekunde za projekte.

load_combat_balance() učitava podatke paketnim putem i odbija nedostajuće objekte, nebrojčane, beskonačne ili nepozitivne vrijednosti.

Projectile čuva položaj, smjer, brzinu, štetu, tim, radijus i preostalo trajanje.

Kretanje koristi **dt**, položaj se omata unutar svijeta, a istekli projektil prestaje biti aktivan. **CircleBody** i **circles_overlap()** provjeravaju dodir kružnica najkraćim putem preko granice svijeta.

PlayerCombatState počinje punim zdravljem, ograničava ga na raspon od nule do maksimuma

i nakon prihvaćene štete aktivira kratku neranjivost. Ponovljeni pogodak tijekom zaštite se odbija, a liječenje vraća samo zdravlje koje nedostaje.

Oružja, protivnici i pickup

WeaponController čuva odabrano primarno oružje te zasebna hlađenja za cannon, spread gun i raketu. Cannon stvara jedan projektil, spread gun tri projektila pod kutovima -0,16, 0 i +0,16 radijana, a raketa koristi neovisnu tipku i hlađenje. Aktivni broj projektila ograničen je na 64.

EnemyKind i **Enemy** definiraju tri zaključane standardne vrste protivnika: scout, gunship i bomber. Svaki protivnik ima vlastiti trup, brzinu, radijus sudara, vrijednost bodova, kontaktnu štetu, projektil, hlađenje napada i proceduralni oblik. Scout je brz i slab, gunship je srednji dvobojni brod, a bomber je spor i izdržljiv s težim projektilom. Kretanje koristi najkraći smjer kroz omotani svijet, a napadi stvaraju projekte tima **enemy**.

WaveDirector učitava **data/waves.json**, provjerava da postoje točno tri vala i stvara protivnike prema konfiguriranim odgodama, udaljenostima ispred kamere i bočnim pomacima. Prvi val uvodi šest scoutova, drugi kombinira scoutove i gunshipove, a treći dodaje bombere. Nakon što su svi spawnovi vala iskorišteni i nema živih protivnika, direktor pokreće kratku stanku i zatim prelazi na sljedeći val. Nakon trećeg vala postavlja stanje **waves_complete**, koje trenutačna borbena sesija koristi za stvaranje bossa.

DreadnoughtBoss predstavlja ISS Goliath. Stvara se ispred kamere nakon završetka trećeg vala, ima 1.250 HP, velik radijus sudara i dvije faze. Prva faza koristi burst od tri projektila i hlađenje od 0,86 s, a druga počinje na 50 % zdravlja, koristi burst od pet projektila i hlađenje od 0,50 s. Boss se održava u prednjem sektoru kako ga igrač ne bi nenamjerno prošao i izgubio iz vidljivog borbenog prostora. Kada Goliath u drugoj fazi prvi put padne na 20 % zdravlja ili manje, **CombatSession** dodjeljuje igraču jednokratni hitni popravak od 50 HP-a, pri čemu **PlayerCombatState.heal()** i dalje ograničava zdravlje na maksimum.

CombatSession jednom po frameu povezuje ulaze, projekte, **WaveDirector**, neprijatelje, kružne sudare, boss susret, repair pickup, zdravlje igrača i bodove. Igračevi projektili uništavaju protivnike i odmah dodaju njihove bodove; uništeni protivnik ostavlja popravak koji vraća do 36 HP-a, dodaje 75 bodova i nestaje nakon 12 sekundi. Uništenje bossa dodaje 5.000 bodova i postavlja **victory**, dok zdravlje igrača na nuli postavlja **game_over**. Poseban flag **boss_critical_repair_awarded** sprječava da se boss-critical popravak na 20 % zdravlja ponovi više puta u istom pokušaju. Nakon terminalnog stanja borbena simulacija i kamera se zaustavljaju, a prozor se i dalje može zatvoriti.

Proceduralne slike razlikuju mjedeni cannon, cijan spread gun, crvenu raketu, neprijateljske

projektile, tri vrste viktorijanskih zračnih brodova, ISS Goliath dreadnought i aether repair ćeliju. **BillboardProjector** ih zajednički sortira i crta. Minimalistički steampunk engleski HUD sada je pognut u gornjoj horizontalnoj traci i prikazuje trup, odabrano oružje, hlađenje rakete, val, bodove, vrijeme pokušaja, broj preostalih protivnika, trenutnu prijetnju, stanje dolaska vala, boss health bar, boss fazu, poruku pobjede ili poraza, brzinu i FPS. Gameplay scena više ne crta dodatni tekst s kontrolama ispod broda.

Telemetry za balansiranje

CombatSession sada čuva osnovne vrijednosti potrebne za ručno balansiranje:

elapsed_time, **enemies_destroyed**, **repairs_collected** i **damage_taken**. Vrijeme se povećava samo dok je borba aktivna; pauza se obrađuje izvan sesije, a nakon pobjede ili poraza **update()** se odmah zaustavlja. Primljena šteta mjeri se iz stvarnog pada zdravlja, pa neranjivost i ograničenje zdravlja ne stvaraju lažne vrijednosti.

Broj uništenih standardnih protivnika povećava se samo kada neprijatelj stvarno prijeđe u neaktivno stanje nakon igračeva pogotka. Broj skupljenih popravaka povećava se pri stvarnom kontaktu s aktivnim pickupom. Pickup ima znatno širi krug skupljanja od sidrišnog radijusa spritea kako bi se popravak i zeleni feedback aktivirali čim igrač uđe u područje velikog perspektivnog prikaza plusa, a ne tek nakon prolaska pokraj njega. Ovi podaci nisu novi gameplay sustav; služe kao mjerni sloj za iduće balance commitove i za ručni obrazac **tests/playtest-notes.md**.

Repair SFX, zeleni vizualni feedback, health i score koriste isti raniji collect događaj. Time se pickup aktivira čim igrač vizualno uđe u područje velikog plavog križa, bez odvojenog audio cuea koji bi mogao kasniti ili se razići od zelenog efekta.

Vizualni feedback

CombatFeedback u **CombatSession** bilježi samo događaje zadnjeg framea: uništene položaje, pozicije skupljenih popravaka, stvarno ispaljena igračeva oružja, vrste neprijateljskih hica, početak novog vala, pojavu bossa, boss pogodak, uništenje bossa, stvarnu štetu nad igračem i terminalna stanja. Ti se podaci brišu na početku svakog **update()** poziva i ne stvaraju zaseban sloj pravila igre.

EffectsState u renderer sloju čuva kratkotrajne svjetske i lokalne efekte. Uništenje standardnog protivnika dodaje malu narančasto-mjedenu eksploziju u svjetskim koordinatama, skupljeni repair pickup dodaje kratki zeleni/cijan plus efekt s većim prstenom, boss pogodak dodaje crveno-cijan spark, stvarno pucanje dodaje muzzle flash ispred Kestrela, a stvarno primljena šteta dodaje lokalni crveni marker pri rubu ekrana. Svjetski efekti koriste postojeći **BillboardProjector**, pa poštuju rotaciju kamere i omatanje svijeta. Efekti se crtaju prije HUD-a, bez screen shakea i bez promjene **Mode7Renderer** ili stanja kamere.

Zvučni efekti

AudioManager učitava stabilno imenovane ElevenLabs MP3 datoteke iz **assets/audio/sfx**. Svaki efekt ima kratak **SoundEffect** identifikator, kontroliranu glasnoću i opcionalni **maxtime** za dulje izvore poput neprijateljskog lakog hica i weapon-ready klika. Ako **pygame.mixer** ne može inicijalizirati audio uređaj, manager se prebacuje u no-op način i gameplay nastavlja bez iznimke.

Glavna petlja povezuje audio s istim **CombatFeedback** događajima koji već pokreću vizualne efekte: pucanje igrača, protivničku paljbu, uništenje protivnika, boss pogodak, boss dolazak, boss uništenje, štetu igrača i skupljanje repair pickupa. UI zvukovi pokrivaju izbor u meniju, pauzu i odabir oružja, a pobjeda i game-over reproduciraju se samo jednom po pokušaju. MP3 datoteke su uključene u **pyproject.toml** package-data i provjeravaju se u **scripts/package.sh**.

Glazbena podloga

Glazba je proceduralno generirana u WAV datoteke unutar **assets/audio/music**.

MusicTrack definira šest loopova: **MENU**, **WAVE_1**, **WAVE_2**, **WAVE_3**, **BOSS_PHASE_1** i **BOSS_PHASE_2**. Svaki loop traje osam sekundi, generira se kao stereo **int16** NumPy matrica kroz **scripts/generate_music_assets.py**, sprema kao PCM WAV i učitava kao **pygame.mixer.Sound** pri inicijalizaciji audio sloja.

Game pušta **MENU** loop u glavnom izborniku i na ekranu uputa. Tijekom aktivnog igranja bira glazbu prema napretku borbe. Prva tri vala imaju zasebne tempo/intenzitet profile, boss faza 1 koristi sporiji prijeteci loop, a boss faza 2 prelazi u brži i gušći loop nakon što Goliath padne na 50 % HP-a ili manje. Glazba se gasi pri pauzi i terminalnim stanjima, dok victory i game-over SFX imaju prioritet nad borbenom glazbom. Volumeni loopova namjerno su viši od prve tehničke verzije, a često ponavljani borbeni SFX-ovi dodatno su stišani kako bi glazba ostala čujna ispod paljbe i eksplozija.

Loopovi su evidentirani u **assets/manifest.csv** i **docs/asset-licenses.csv** kao projektno generirani resursi. Testovi provjeravaju da je svaka glazbena tema definirana, da WAV datoteka postoji, da ima valjanu glasnoću i da generator vraća stereo **int16** signal s vidljivim uzorkom.

Aplikacijska stanja

AppState odvaja aplikacijski tok od borbene simulacije. Početno stanje je **MAIN_MENU**, zbog čega se borba više ne pokreće odmah pri otvaranju prozora. **Enter** ili **Space** stvaraju novu kameru i novu **CombatSession** te prelaze u **PLAYING**, dok **I** otvara **INSTRUCTIONS**. U uputama se **Enter** ili **Space** koriste za početak misije, a **M** ili **Esc** vraćaju glavni izbornik.

Tijekom **PLAYING** stanja **Esc** prelazi u **PAUSED**. Dok je igra pauzirana, kamera, projektili, protivnici, boss i cooldowni se ne ažuriraju; crta se posljednji zamrznuti frame s pauznim panelom. Iz pauze **Esc** nastavlja igru, **R** stvara novi pokušaj, a **M** vraća izbornik.

Pobjeda i poraz ostaju terminalna stanja unutar **CombatSession**, jer ovise o pravilima borbe. Kada su **victory** ili **game_over** aktivni, **PLAYING** crta završni panel s konačnim scoreom. **R** tada resetira misiju, a **M** resetira stanje i vraća glavni izbornik.

Kamera i vrijeme

Camera čuva položaj (**x**, **y**), smjer u radianima i brzinu. Svaki frame prima proteklo vrijeme **dt**, rotaciju i gas. Smjer se normalizira na jedan puni krug, brzina se ograničava na 20-92 jedinice u sekundi, a položaj se omata unutar svijeta veličine 2.048 jedinica.

Tipke **A/D** ili lijevo/desno daju os rotacije, a **W/S** ili gore/dolje daju os gasa. Kretanje koristi **dt**, zbog čega prijedena udaljenost ne ovisi o broju frameova u sekundi.

Pokretanje i provjera

```
source .venv/bin/activate
python -m aetherfront
./scripts/validate.sh
python scripts/benchmark_mode7.py --duration 60 --minimum 55
./scripts/package.sh
open dist/Aetherfront.app
```

Neobvezni argument **Game.run(max_frames=...)** postoji samo za ograničeno izvođenje u automatiziranim testovima. U normalnom pokretanju aplikacija radi do zatvaranja prozora.

package.sh prvo izvodi Ruff i Pytest, zatim gradi prozorsku aplikaciju s identifikatorom **hr.foi.aetherfront**. PyInstallerova konfiguracija i cache usmjereni su u ignoriranu mapu **tmp/**, a skripta završava pogreškom ako izvršna datoteka ili paketni **balance.json** ili **waves.json** ili osnovni SFX asset unutar **.app** paketa ne postoje. Nakon uspješnog **.app** builda skripta izrađuje **dist/Aetherfront-Zeppelin-Wars-macOS.zip**, raspakirava ga u privremenu mapu i provjerava da raspakirani ZIP sadrži izvršnu aplikaciju. Rani ARM64 paket 22. lipnja uspješno je izgrađen, ostao aktivan u headless smoke testu i stvarno se otvorio i zatvorio u macOS-u.

Mode7 benchmark sada tijekom 60-sekundnog izvođenja ispisuje napredak svakih 10 sekundi i uvijek šalje završni progress signal. Prekid tipkama **Ctrl+C** vraća izlazni kod 130 i kratku poruku bez Python tracebacka.

Sljedeći tehnički korak

Završno paketiranje, ZIP smoke test i PDF export glavnih dokumenata provedeni su 2. srpnja 2026. Prezentacija se izrađuje naknadno prema korisnikovoj odluci.